NAMA : DELA SHINTA SUMOMBA

NIM : H061241065

METODE INVERSI

BAB 5 Inversi Non-Linier

5.1 Parameterisasi Pemodelan

Parameterisasi pemodelan merupakan proses menentukan variabel yang digunakan untuk menyatakan data dan parameter model. Pemilihan parameterisasi berpengaruh terhadap hasil inversi karena perubahan variabel dapat mengubah bentuk hubungan antara data dan model. Pada persoalan tertentu, transformasi parameter dapat menyederhanakan hubungan yang rumit menjadi lebih mudah dihitung. Namun, transformasi juga dapat mengubah asumsi statistik kesalahan data. Oleh karena itu, parameterisasi perlu dipilih dengan mempertimbangkan sifat data dan model yang digunakan.

Buku memperlihatkan bahwa regresi dapat memberikan hasil berbeda ketika variabel yang dianggap bebas dan variabel yang dianggap terikat dipertukarkan. Hal ini terjadi karena kesalahan statistik diperlakukan pada variabel yang berbeda. Jadi, pemilihan variabel bukan sekadar persoalan matematis, tetapi juga berkaitan dengan asumsi kesalahan pengukuran.

5.2 Inversi Non-Linier dengan Pendekatan Linier

Hubungan antara data pengamatan dan parameter model pada inversi non-linier dinyatakan melalui fungsi pemodelan ke depan, yaitu d = g(m). Penyelesaian dilakukan dengan melinearkan fungsi tersebut di sekitar model awal menggunakan ekspansi Taylor orde pertama. Model yang dicari dinyatakan sebagai model awal ditambah perturbasi atau koreksi parameter.

$$m = m_0 + \Delta m$$

Linearisasi menghasilkan matriks Jacobi (Jacobian), yang elemennya merupakan turunan parsial respons data terhadap parameter model. Matriks ini menunjukkan sensitivitas data terhadap perubahan masing-masing parameter.

$$J_{ij} = \partial g_i / \partial m_j$$

Selisih antara data pengamatan dan respons model awal digunakan untuk menghitung koreksi parameter. Setelah model diperbarui, respons model dihitung kembali dan proses diulang. Salah satu bentuk pembaruan model pada metode Gauss-Newton adalah:

$$m_{n+1} = m_n + (J_n{}^T J_n)^{-1} J_n{}^T [d - g(m_n)]$$

Proses iterasi dihentikan ketika misfit sudah cukup kecil atau perubahan model tidak lagi berarti. Buku juga membahas bentuk inversi berbobot, inversi untuk kasus jumlah data dan

parameter yang berbeda, serta inversi teredam. Faktor redaman ditambahkan untuk membantu menghindari solusi yang tidak stabil.

5.3 Metode Newton

Metode Newton digunakan untuk mencari nilai minimum fungsi objektif dengan memanfaatkan turunan pertama dan turunan kedua. Fungsi objektif menyatakan ukuran ketidakcocokan antara data pengamatan dan data hasil perhitungan model. Turunan kedua membentuk matriks Hessian yang memberikan informasi tentang kelengkungan fungsi objektif di sekitar model yang sedang diuji.

Perhitungan Hessian dapat menjadi rumit. Oleh karena itu, digunakan pendekatan Gauss-Newton yang menyederhanakan metode Newton dengan mengabaikan suku turunan orde kedua tertentu. Penyederhanaan ini membuat perhitungan lebih praktis, tetapi proses iterasi tetap perlu diawasi agar tidak mengalami ketidakstabilan atau gagal konvergen.

5.4 Metode Gradien

Metode gradien memperbaiki model dengan bergerak ke arah negatif gradien fungsi objektif, yaitu arah penurunan kesalahan paling cepat secara lokal. Besarnya perubahan parameter ditentukan oleh ukuran langkah (step size). Jika langkah terlalu besar, proses dapat melewati daerah solusi; jika terlalu kecil, konvergensi menjadi lambat.

$$\Delta m_{\mathrm{n}} = -k[\partial E/\partial m]_{\mathrm{m=mn}}$$

Metode gradien relatif sederhana, tetapi umumnya membutuhkan iterasi lebih banyak dibandingkan Gauss-Newton. Metode Levenberg-Marquardt menggabungkan karakteristik metode gradien dan Gauss-Newton melalui faktor redaman. Redaman disesuaikan selama iterasi agar proses pencarian solusi menjadi lebih stabil.

5.5 Keterbatasan Pendekatan Linier

Pendekatan linier hanya menggambarkan hubungan data dan model di sekitar model yang sedang digunakan. Karena itu, hasil inversi sangat dipengaruhi oleh pemilihan model awal. Jika model awal terlalu jauh dari solusi, koreksi parameter dapat bergerak ke arah yang kurang tepat atau proses iterasi tidak konvergen.

Fungsi objektif juga dapat memiliki beberapa minimum lokal. Metode yang menggunakan informasi lokal berisiko berhenti pada minimum lokal dan belum tentu mencapai minimum global. Selain itu, matriks yang singular atau hampir singular dapat menyebabkan koreksi model terlalu besar dan perhitungan tidak stabil. Penggunaan informasi awal (a priori),

pemilihan model awal yang wajar, pengujian beberapa model awal, serta teknik seperti SVD dapat membantu mengurangi masalah tersebut.

BAB 9 Aplikasi Inversi Non-Linier Dengan Pendekatan Linier

9.1 Penentuan Episenter Gempa

Penentuan episenter gempa dilakukan dengan menggunakan waktu tiba gelombang P yang tercatat di beberapa stasiun pengamatan. Posisi stasiun diketahui, sedangkan koordinat episenter merupakan parameter yang dicari. Waktu tempuh gelombang bergantung pada jarak antara episenter dan stasiun serta kecepatan gelombang P. Hubungan jarak dan waktu tempuh tersebut bersifat non-linier.

$$t_i^{cal} = t_0 + r_i/V_p$$

Koordinat episenter diperbaiki secara iteratif dengan membandingkan waktu tiba hasil perhitungan dan waktu pengamatan. Matriks Jacobi menunjukkan sensitivitas waktu tempuh terhadap perubahan koordinat episenter. Pada contoh sintetik dalam buku, posisi episenter sebenarnya adalah (40, 30) km. Pengujian beberapa model awal memperlihatkan bahwa iterasi dapat menuju posisi yang mendekati koordinat sebenarnya, disertai penurunan nilai kesalahan RMS.

Code Python

In [ ]:
```python
import numpy as np
import matplotlib.pyplot as plt

# Data posisi stasiun (km)
stasiun = np.array([
    [20, 10],
    [50, 25],
    [40, 50],
    [10, 40]
], dtype=float)

# Waktu tiba gelombang P (detik)
waktu_observasi = np.array([7.1, 2.8, 5.0, 7.9])

# Kecepatan gelombang P (km/detik)
vp = 4.0

# Tiga model awal
model_awal = [
    (30, 10),
    (10, 10),
    (5, 45)
]

def inversi_episenter(model_awal, maksimum_iterasi=4):
    model = np.array(model_awal, dtype=float)
    lintasan = [model.copy()]
```

```
    for i in range(maksimum_iterasi):

        # Menghitung jarak episenter ke setiap stasiun
        dx = model[0] - stasiun[:, 0]
        dy = model[1] - stasiun[:, 1]
        jarak = np.sqrt(dx**2 + dy**2)

        # Menghitung waktu tiba teoritis
        waktu_hitung = jarak / vp

        # Membentuk matriks Jacobi
        jacobi = np.column_stack((
            dx / (vp * jarak),
            dy / (vp * jarak)
        ))

        # Menghitung koreksi model
        koreksi, *_ = np.linalg.lstsq(
            jacobi,
            waktu_observasi - waktu_hitung,
            rcond=None
        )

        # Memperbarui posisi episenter
        model = model + koreksi
        lintasan.append(model.copy())

    return np.array(lintasan)


# Menghitung ketiga lintasan
semua_lintasan = [
    inversi_episenter(m0) for m0 in model_awal
]

# Menampilkan hasil iterasi
for nomor, lintasan in enumerate(semua_lintasan, start=1):
    print(f"\nModel awal {nomor}")
    print("Iterasi    x0 (km)    y0 (km)")

    for i, posisi in enumerate(lintasan):
        print(f"{i:>4}      {posisi[0]:>8.2f}   {posisi[1]:>8.2f}")


# Membuat grafik Gambar 9.1
fig, ax = plt.subplots(figsize=(7, 6))

# Menggambar lintasan iterasi
for lintasan in semua_lintasan:
    ax.plot(
        lintasan[:, 0],
        lintasan[:, 1],
        '-o',
        color='black',
        linewidth=1.3,
        markersize=3.5
    )

# Menggambar posisi stasiun
```

```
ax.scatter(
    stasiun[:, 0],
    stasiun[:, 1],
    marker='^',
    color='black',
    s=55
)

# Memberi label stasiun
for i, (x, y) in enumerate(stasiun, start=1):
    ax.text(x + 0.5, y + 1, f'st-{i}', fontsize=10)

# Episenter sebenarnya (40, 30)
episenter = (40, 30)

# Lingkaran dengan jari-jari 2 km
lingkaran = plt.Circle(
    episenter, 2,
    fill=False,
    color='black'
)
ax.add_patch(lingkaran)

# Menandai titik episenter
ax.plot(40, 30, 'o',
        markerfacecolor='none',
        markeredgecolor='black',
        markersize=9)

# Pengaturan grafik
ax.set_xlim(0, 60)
ax.set_ylim(0, 60)
ax.set_xticks(np.arange(0, 61, 10))
ax.set_yticks(np.arange(0, 61, 10))
ax.set_xlabel('x0 (km)')
ax.set_ylabel('y0 (km)')
ax.set_aspect('equal')
ax.tick_params(direction='out', top=True, right=True)

plt.tight_layout()
plt.show()
```

Model awal 1

Iterasi	x0 (km)	y0 (km)
0	30.00	10.00
1	44.17	24.54
2	39.76	30.03
3	40.02	30.06
4	40.02	30.06

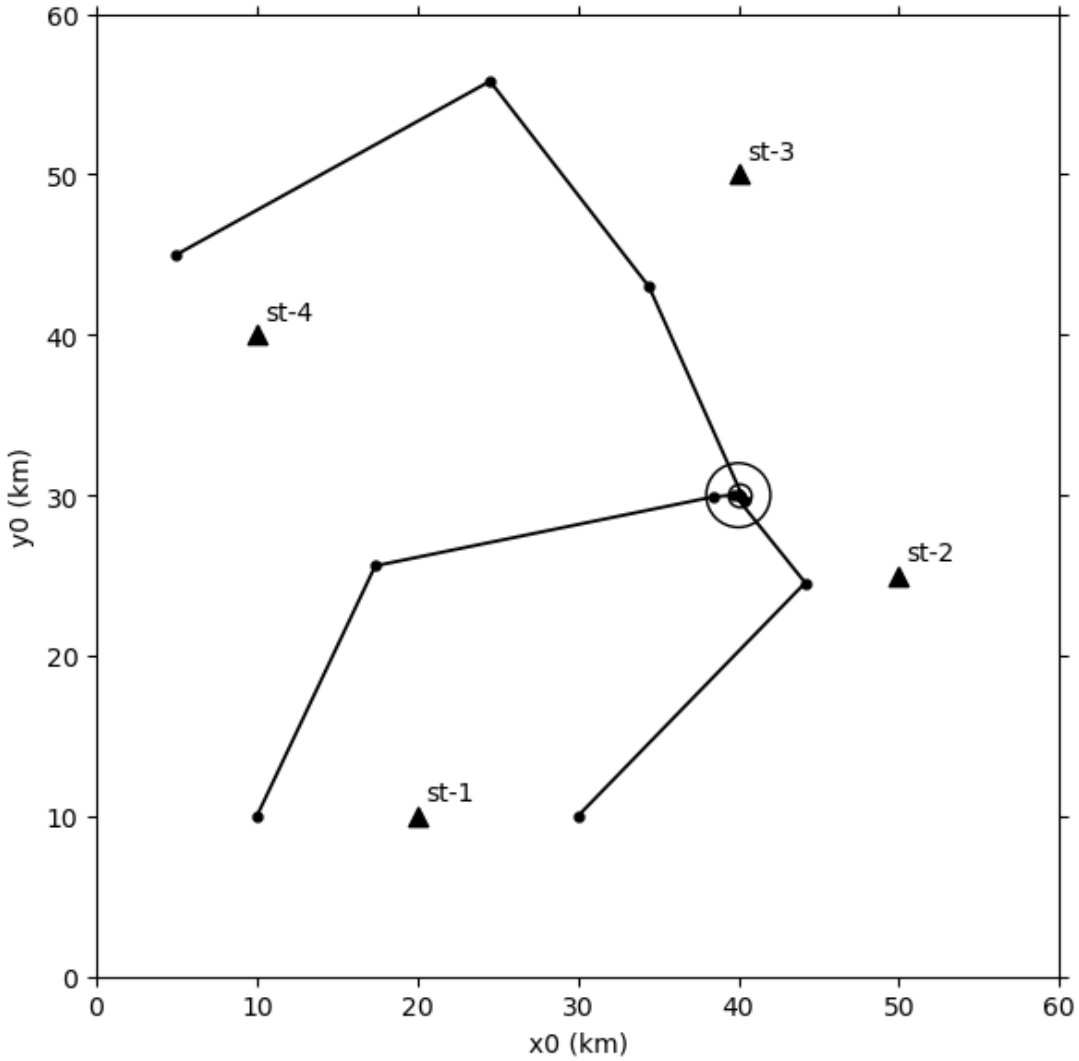
Model awal 2

Iterasi	x0 (km)	y0 (km)
0	10.00	10.00
1	17.38	25.60
2	38.46	29.90
3	40.02	30.04
4	40.02	30.06

Model awal 3

Iterasi	x0 (km)	y0 (km)

```
0       5.00    45.00
1      24.53    55.80
2      34.43    42.99
3      40.37    29.68
4      40.03    30.06
```



Program Python tersebut digunakan untuk menentukan posisi episenter gempa berdasarkan waktu tiba gelombang P pada empat stasiun pengamatan. Perhitungan dilakukan menggunakan metode Gauss-Newton dengan tiga model awal yang berbeda. Pada setiap iterasi, program menghitung waktu tiba teoritis, membentuk matriks Jacobi, dan memperbarui koordinat episenter berdasarkan selisih data pengamatan dan hasil perhitungan. Hasilnya menunjukkan bahwa ketiga model awal mengalami konvergensi menuju posisi sekitar (40, 30) km, sesuai dengan contoh pada buku.

9.2 Inversi Data Geolistrik 1-D

Inversi geolistrik 1-D menggunakan asumsi bahwa bawah permukaan tersusun atas lapisan-lapisan horizontal. Setiap lapisan memiliki parameter resistivitas dan ketebalan, sedangkan

lapisan paling bawah dianggap sebagai medium dengan ketebalan tak hingga. Data yang digunakan berupa resistivitas semu yang diperoleh dari pengukuran dengan variasi jarak elektroda.

$$M = 2n - 1$$

Persamaan tersebut menyatakan jumlah parameter model untuk n lapisan, yaitu n resistivitas dan n−1 ketebalan. Hubungan antara resistivitas semu dan parameter lapisan bersifat non-linier. Matriks Jacobi dihitung menggunakan pendekatan beda hingga, yaitu dengan mengubah satu parameter sedikit demi sedikit lalu menghitung perubahan respons model.

Untuk menjaga kestabilan perhitungan, digunakan faktor redaman dan Singular Value Decomposition (SVD). Pada contoh sintetik dalam buku, hasil inversi memberikan misfit sekitar 4,2% dan 4,9%, mendekati tingkat noise data sebesar 5%. Contoh ini menunjukkan bahwa model inversi dapat mendekati model sebenarnya apabila model awal dan proses perhitungan dipilih dengan baik.

9.3 Inversi Data Magnetotellurik 2-D

Metode magnetotellurik (MT) memanfaatkan medan elektromagnetik alami untuk memperkirakan distribusi resistivitas bawah permukaan. Data yang digunakan antara lain resistivitas semu dan fase sebagai fungsi periode atau frekuensi. Pada pemodelan 2-D, resistivitas berubah dalam arah horizontal dan vertikal, sehingga bawah permukaan direpresentasikan sebagai blok-blok dengan nilai resistivitas.

Pemodelan ke depan didasarkan pada persamaan Maxwell. Data MT dapat dianalisis menggunakan dua polarisasi, yaitu TE dan TM, yang memiliki komponen medan listrik dan magnet berbeda. Matriks Jacobi dihitung dengan pendekatan beda hingga karena proses pemodelan ke depan cukup kompleks.

Jumlah parameter model pada MT 2-D dapat lebih banyak daripada jumlah data. Oleh karena itu, inversi memerlukan regularisasi agar model yang diperoleh tetap stabil. Salah satu caranya adalah menerapkan kendala kehalusan spasial, yaitu membatasi perubahan resistivitas antarblok yang berdekatan. Data TE dan TM saling melengkapi; penggunaan keduanya dapat memberikan gambaran anomali resistivitas yang lebih baik dibandingkan penggunaan satu mode saja.